Large-Scale CMG Benchmarks

Rust versus the official MATLAB/C MEX package

CMG maintainers

August 2026

Abstract

This report compares the current Rust Combinatorial Multigrid (CMG) package with the official MATLAB solver and its C MEX kernels at upstream commit `19752fc102f8cae8e34f66457bfaccb1aaa60375`. It separately reports CMG hierarchy construction, stationary preconditioner application, single-right-hand-side preconditioned conjugate gradient (PCG), setup-plus-solve time, input assembly, memory, CPU scaling, and a compact sixteen-right-hand-side supplement. At 32 application CPUs, Rust required a geometric-mean 0.265 times the MATLAB setup time and 0.666 times the setup-plus-solve time across the 15 main graph/size cases; its process peak RSS was 0.150 times MATLAB's. Scaling from 1 to 32 CPUs was modest for both packages, so the largest cross-package gains arise mainly from lower setup cost and memory use rather than near-linear CPU scaling. The standalone pinned-C measurements are intentionally limited to hot kernels and a small recursive-cycle comparison; they are not described as an end-to-end C solver.

1 Scope and interpretation

The primary question is how the two maintained user-facing implementations behave as graph size, sparsity, topology, and application CPU count change. The comparison follows the official MATLAB workflow: first call `cmg_precondition`, then pass its function handle to MATLAB `pcg`. Rust uses its benchmark-only driver and package-owned parallel executor; no public Rust library API was added or changed.

All primary timing figures show medians of three measured repetitions following one warm-up. They are a compact qualification study, not a claim of formal uncertainty bounds. Queue waiting time is excluded. Unsupported, timed-out, memory-failed, or nonconverged cases would remain in the raw archive and be marked in plots; validated production points shown here passed both scheduler and application checks.

2 Graph families and deterministic inputs

For each graph task, the Rust generator writes one canonical little-endian binary edge list plus a bundle of deterministic solution vectors and right-hand sides. MATLAB and Rust read identical bytes. Task receipts record SHA-256 hashes for the graph, right-hand sides, truth vectors, and metadata. A second independent generation was byte-compared during the smoke phase.

Family	Construction and intended characteristic
Weighted path	A connected chain with deterministic positive weights; average degree approaches two.
Two-dimensional grid	A nearly square row-major grid with deterministic positive weights; average degree approaches four.
Worker-firm, degree 3	A balanced bipartite graph in which each worker has three deterministic firm links.
Worker-firm, degree 16	The same bipartite construction with sixteen worker links, yielding roughly eight million canonical edges at one million vertices.
Weak community	Sixteen grid-like blocks connected by one chain bridge between successive blocks.

The main grid uses 100,000, 300,000, and 1,000,000 vertices for every family. Primary plots use vertices on the horizontal axis; a companion figure uses canonical edge count so density differences are visible directly.

3 Stage definitions and accuracy conventions

`preconditioner_setup`
: CMG hierarchy and terminal factor construction only.

`parallel_plan_setup`
: Rust-only construction of the reusable parallel execution plan, reported separately.

`preconditioner_apply`
: One stationary CMG application using an existing hierarchy and workspace. Fast applications are looped for at least one second and normalized per call.

`pcg_solve`
: One PCG solve using a reused preconditioner. The batch supplement reports a sixteen-RHS timing block and normalized per-RHS throughput.

`solver_total`
: Fresh preconditioner setup, Rust parallel-plan setup when applicable, and a PCG solve, with the matrix already resident.

Input and assembly
: Binary input load and graph/sparse-matrix assembly are supplementary and excluded from `solver_total`.

Both implementations use tolerance 10^{-8} and at most 1,000 iterations while preserving their native stopping logic. Every result also records iterations, an independently recomputed relative residual, backward error, and a gauge-invariant scaled difference from the deterministic reference solution. Across accepted points, the maximum independently computed relative residual is 6.76e-08 and the maximum backward error is 9.95e-09.

4 Runtime environment and provenance

Item	Recorded value
SCC project	`welfgr`; isolated root `/projectnb/welfgr/cmg-benchmarks`
Allocated CPU type	Intel Xeon Gold 6242; 32-slot whole-node binding
Observed hosts	scc-eb2, scc-ec4
Observed CPU string	Intel(R) Xeon(R) Gold 6242 CPU @ 2.80GHz
Rust compiler	Rust 1.98.0, project-local minimal toolchain plus rustfmt/clippy
Rust build	`cargo build -release -locked -all-features`; no native CPU flags
MATLAB	2026a; default MEX compiler and `mex -largeArrayDims`
TeX	TeX Live 2023
Post-processing	Python 3.12.2 with Matplotlib 3.8.4
Source identity	`a28f80223220cebfb1c6c456810c6902c444079c+dirty:811130532f76dae3e5173c031f98ba8f308d51ff707af8ce2534a1666e651022`
Environment identity	`07ead3e24ee94352c0410f6a9a2e909df054afb4ebec222b381429478c7a59f4`
Run identity	20260824T152234Z-a28f802-8111305

Each production array task requested `-P welfgr`, `-pe omp 32`, `cpu_type=Gold-6242`, `mem_per_core=8G`, a two-hour limit, and whole-node binding. Concurrency was capped at two simultaneous homogeneous allocations. Application thread limits were 1, 2, 4, 8, 16, and 32; order was rotated by task and Rust/MATLAB order alternated. MATLAB's applied `maxNumCompThreads` limit is recorded in every row. The largest observed process peak RSS was 7.00 GiB. SGE `qacct` records, `/usr/bin/time -v` process receipts, logs, raw repetitions, source manifests, and input hashes remain in the isolated run archive.

The immutable main-run archive was retained even though its Rust and standalone-C rows contained `source_commit=unknown`: the build wrapper exported the source identity under a different environment-variable name than those two compiled drivers expected. MATLAB rows, source manifests, deployed-file hashes, and build logs already carried the exact identity. A reproducible derived copy changes only that field in 96 JSON files from run 20260824T152234Z-a28f802-8111305. Its receipt records the raw-tree digest plus every before/after file hash, and the strengthened validator rejects any timing, numerical, input, or environment change. All findings below use the validated derived copy; the original archive remains byte-for-byte unchanged.

5 Size and density results

Across the 15 matched 32-CPU size/family points, the geometric-mean Rust-to-MATLAB time ratios are 0.265 for CMG setup, 0.790 for one stationary application, 0.783 for reused-preconditioner PCG, and 0.666 for setup plus solve. Thus setup is the clearest aggregate advantage; solve time also favors Rust, but by less. The corresponding geometric-mean process peak-RSS ratio is 0.150. At one million vertices, Rust's setup-plus-solve time is lower for every family: the gap is smallest for the path and largest for the dense worker-firm graph, where the medians are 6.73 seconds for Rust and 14.8 seconds for MATLAB. These aggregates summarize heterogeneous cases and should be read together with Figures 1–3, not as a substitute for them.

All 15 main cases converged under each package's native stopping rule. Rust's maximum reference-solution scaled error was 4.45e-03, compared with 8.16e-04 for MATLAB; these solution-difference diagnostics are more sensitive to the Laplacian nullspace and conditioning than the backward error and independently recomputed residual. The cross-package hierarchy had identical intermediate level sizes but a one-vertex terminal-level offset in 12 cases; the three dense worker-firm cases matched exactly. The official hierarchy returned a nonzero status flag at 3 of the 15 reported 32-CPU points, all from the dense family. Those warnings remain visible and are not converted into silent exclusions.

6 Memory and work diagnostics

7 CPU scaling

For one-million-vertex setup-plus-solve, the geometric-mean 32-versus-1-CPU speedup across graph families is 1.05 for Rust and 1.15 for MATLAB. These are only 5 and 15 percent aggregate reductions in elapsed time, respectively, and several individual curves are nonmonotone. Application CPUs therefore should not be treated as a proportional speed multiplier for the complete workflow. The stage- and family-specific paths below show where parallel overhead or sequential setup dominates.

Figures 8–12 show time against application CPU count for every graph family. Color denotes dataset size; solid lines are Rust and dashed lines are MATLAB. A decrease indicates useful parallel scaling. Setup stages can remain dominated by sequential hierarchy logic even when apply and solve kernels parallelize.

8 Repeated right-hand sides and isolated C kernels

Rust uses the package's memory-bounded across-RHS executor for this supplement; the official MATLAB workflow invokes `pcg` sequentially for each right-hand side while allowing each invocation the requested MATLAB computational-thread limit. The scheduled SCC batch supplement was still pending at report snapshot time, so no repeated-RHS performance conclusion is made here. When results are present, the figure compares the native supported repeated-RHS workflows rather than forcing identical internal scheduling.

The C harness compiles numerical kernels copied from the pinned upstream source and checks every output against Rust. It uses the existing harness's own path and worker-firm formulas rather than the shared end-to-end binary fixtures. Across the large path and worker-firm cases, Rust-to-C SpMV ratios range from 0.666 to 0.915; restriction and prolongation are near parity. For the bounded recursive-cycle cases, the Rust-to-C ratio ranges from 0.737 to 0.803. These timings isolate implementation overhead in SpMV, restriction/prolongation, and a small recursive stationary cycle. They omit the official MATLAB hierarchy constructor, sparse-matrix assembly, MATLAB PCG, and end-to-end solver orchestration; therefore they must not be read as "Rust versus a complete C solver."

9 Validation and acceptance

The smoke phase required successful MEX compilation, deterministic input hashes, numerical certification, and valid SGE accounting. Production acceptance then required, for every task:

- SGE accounting with `failed=0` and `exit_status=0`;
- an application success receipt and a complete, nonduplicated implementation/thread grid;
- finite timing and RSS fields tied to the correct source, environment, and input hashes;

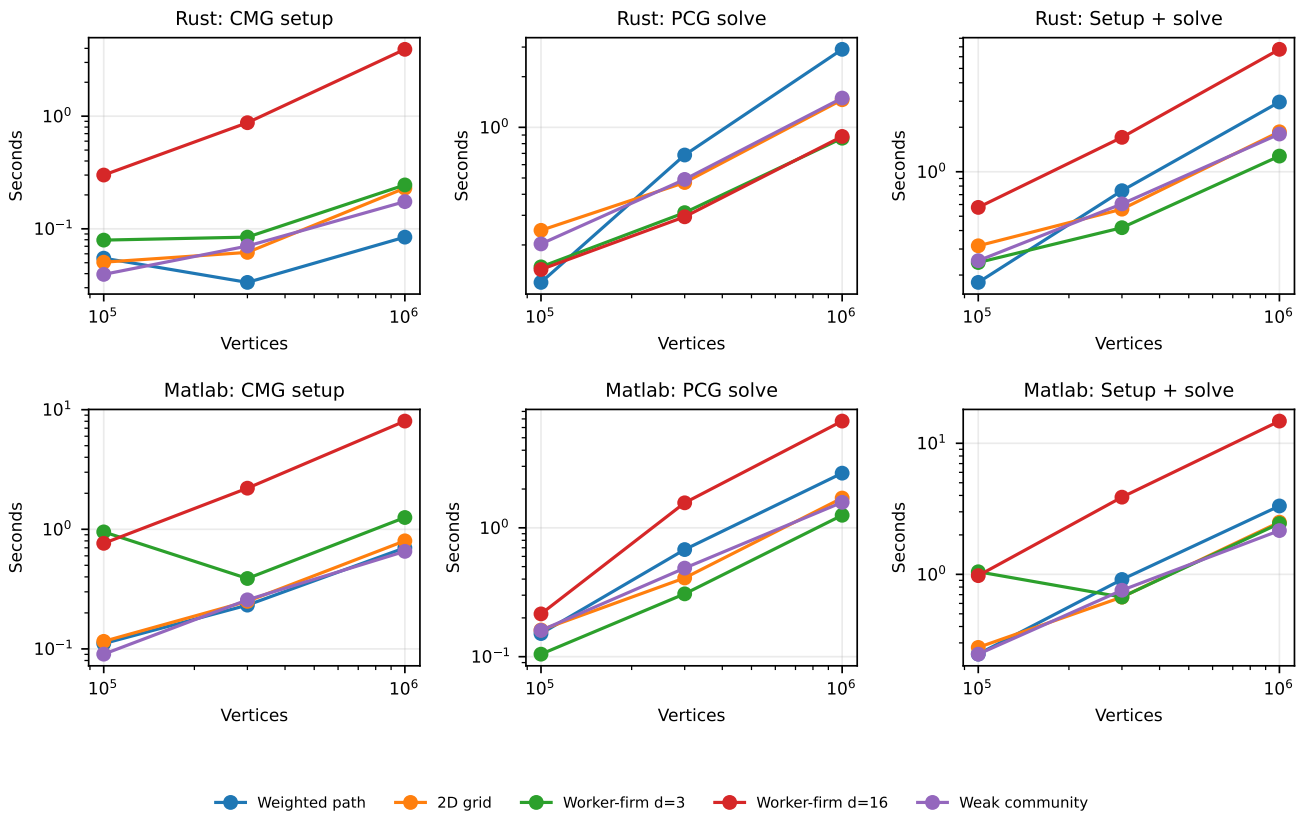

Figure 1: Primary size scaling at 32 application CPUs. Both axes are logarithmic.

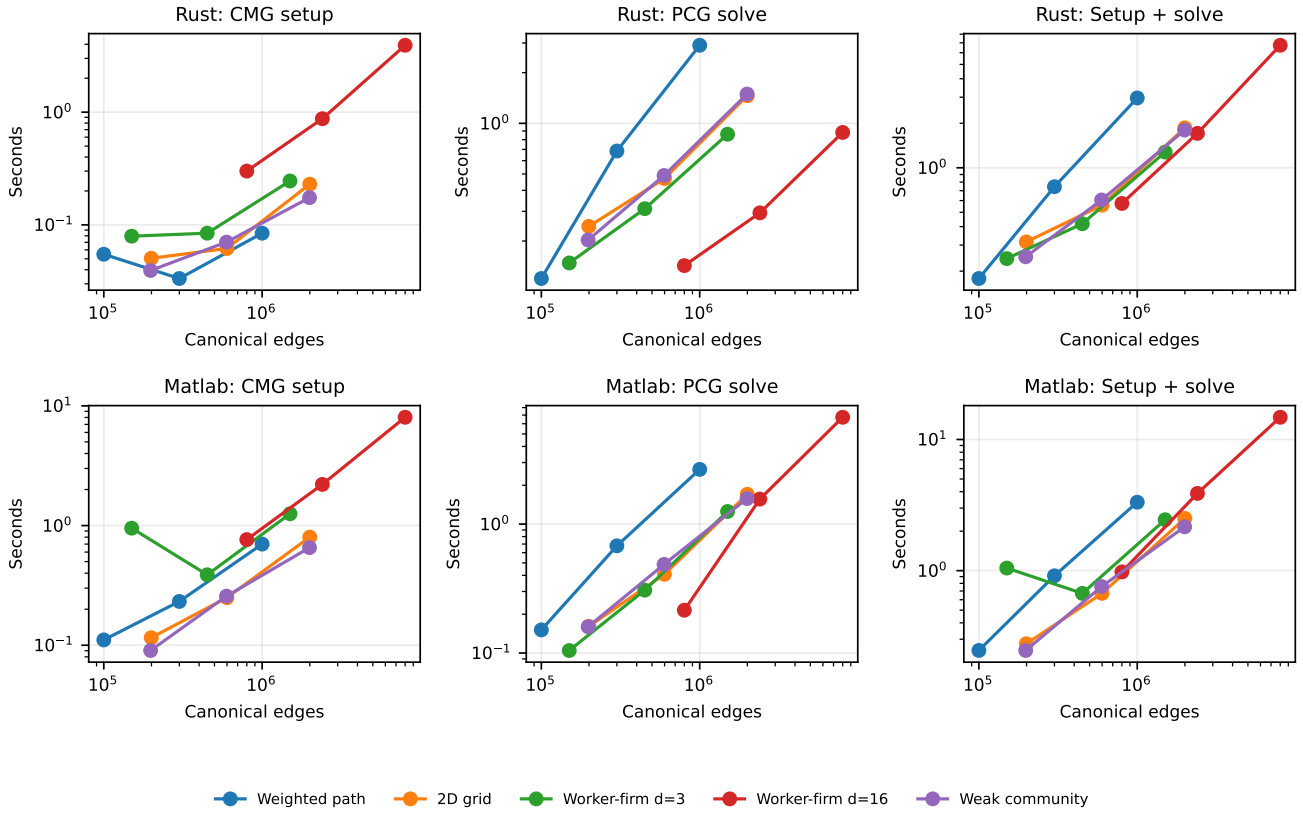

Figure 2: The same primary stages against canonical edge count.

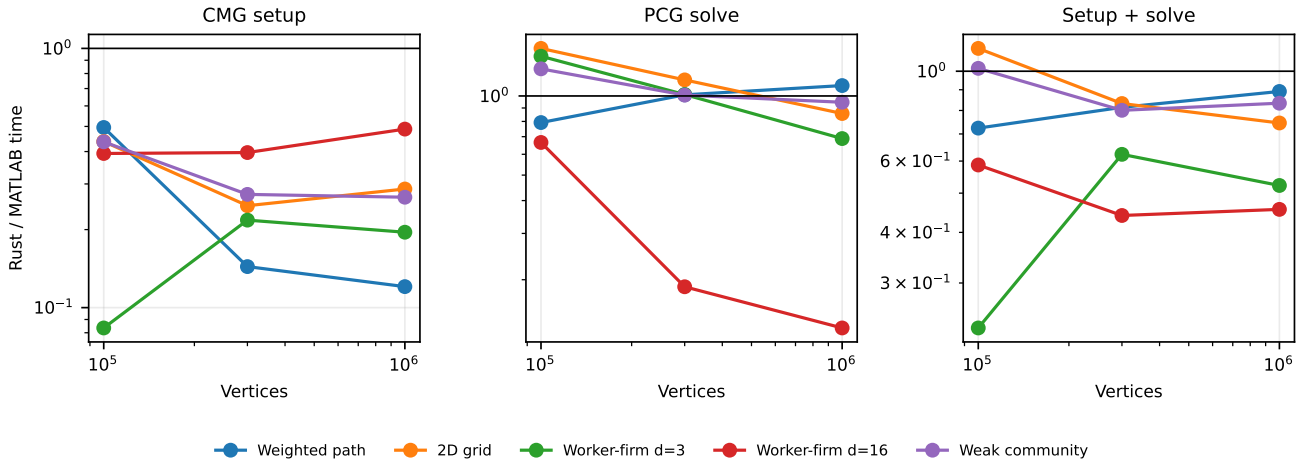

Figure 3: Rust-to-MATLAB median time ratios at 32 application CPUs. Values below one favor Rust.

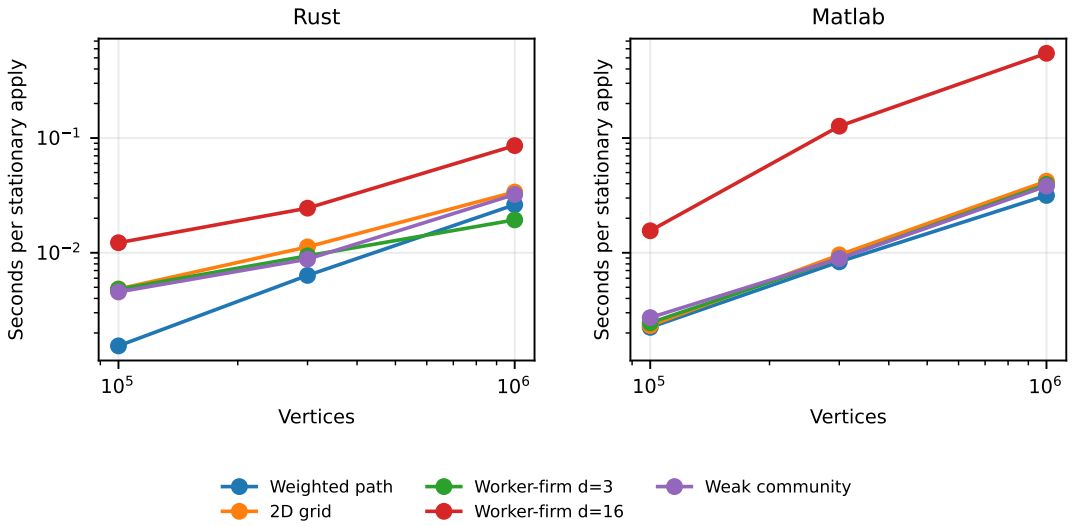

Figure 4: Normalized stationary CMG preconditioner-application time at 32 application CPUs.

Table 2: One-million-vertex results at 32 application CPUs.

Family	Implementation	Vertices	Edges	Iterations	Total (s)
Weighted path	Rust	1,000,000	999,999	53	2.96
Weighted path	Matlab	1,000,000	999,999	62	3.32
2D grid	Rust	1,000,000	1,998,000	26	1.86
2D grid	Matlab	1,000,000	1,998,000	29	2.5
Worker-firm d=3	Rust	1,000,000	1,499,996	20	1.28
Worker-firm d=3	Matlab	1,000,000	1,499,996	23	2.44
Worker-firm d=16	Rust	1,000,000	7,999,978	9	6.73
Worker-firm d=16	Matlab	1,000,000	7,999,978	11	14.8
Weak community	Rust	1,000,000	1,992,015	26	1.8
Weak community	Matlab	1,000,000	1,992,015	29	2.16

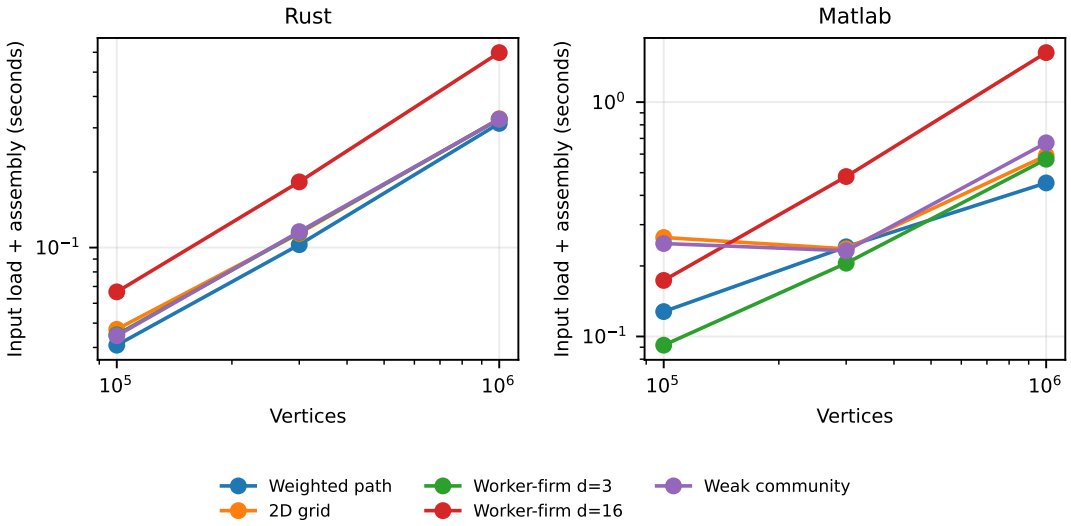

Figure 5: Supplementary binary input load plus graph or sparse-matrix assembly time. This stage is excluded from setup-plus-solve.

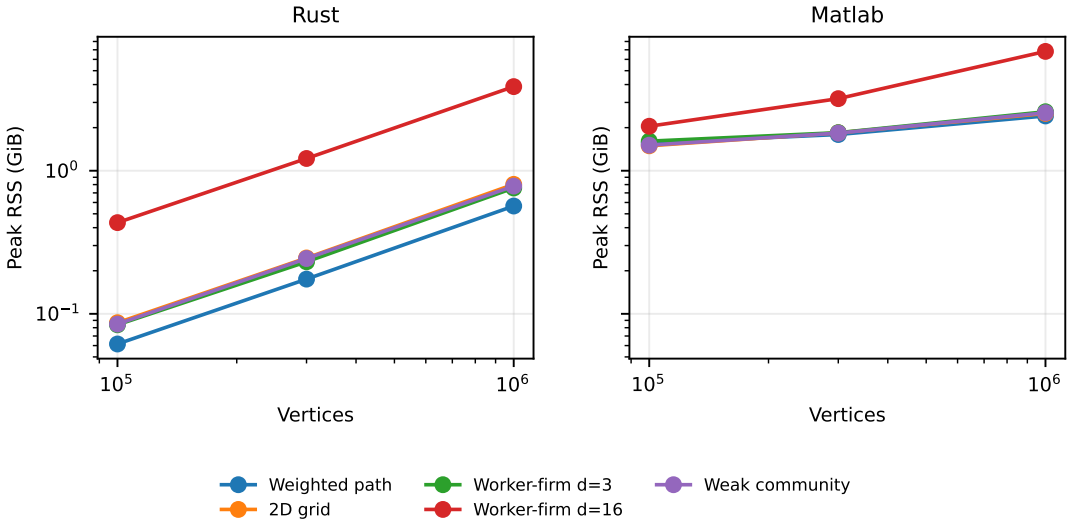

Figure 6: Peak resident set size from `/usr/bin/time -v`. The measure includes each process's input and runtime storage.

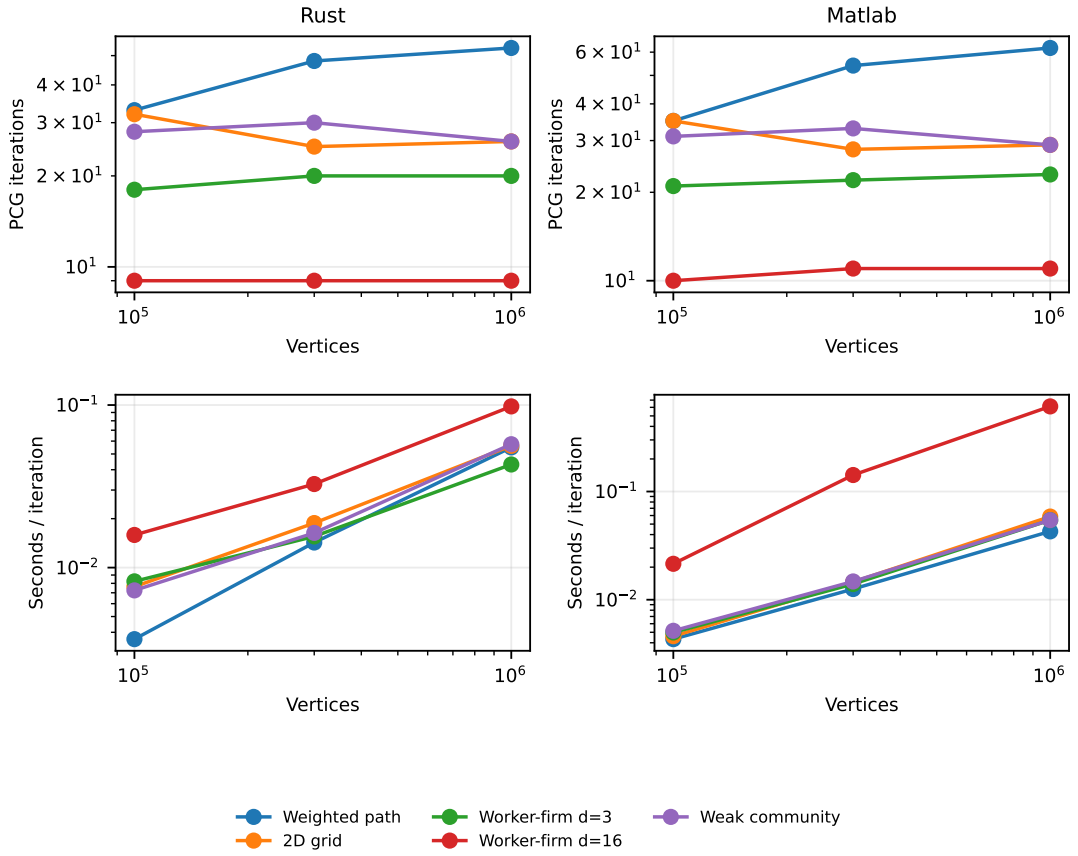

Figure 7: PCG work diagnostics at 32 CPUs. Iteration count separates convergence behavior from time per iteration.

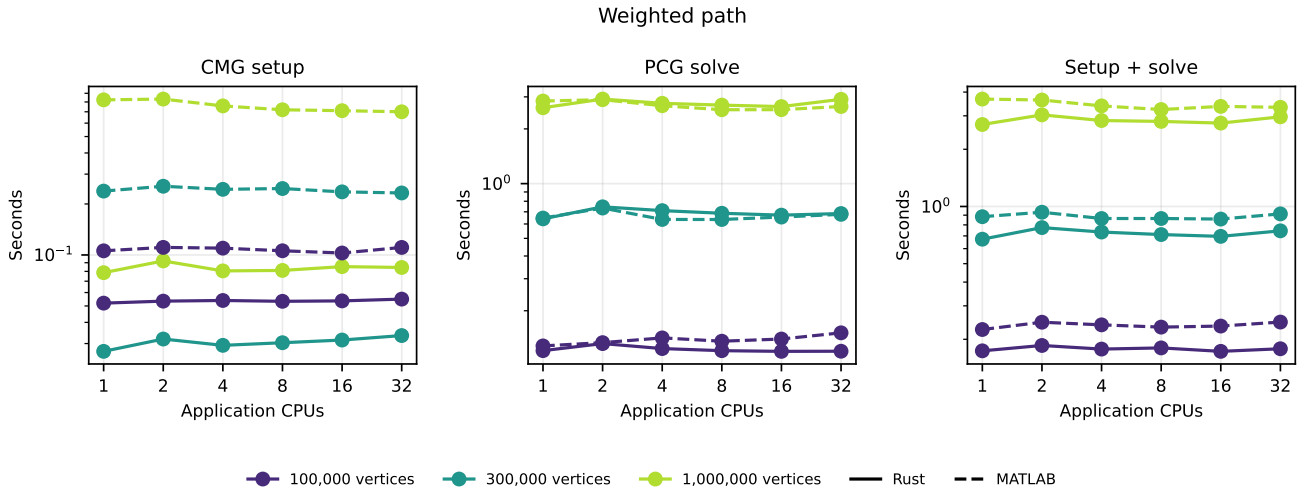

Figure 8: CPU scaling: weighted path.

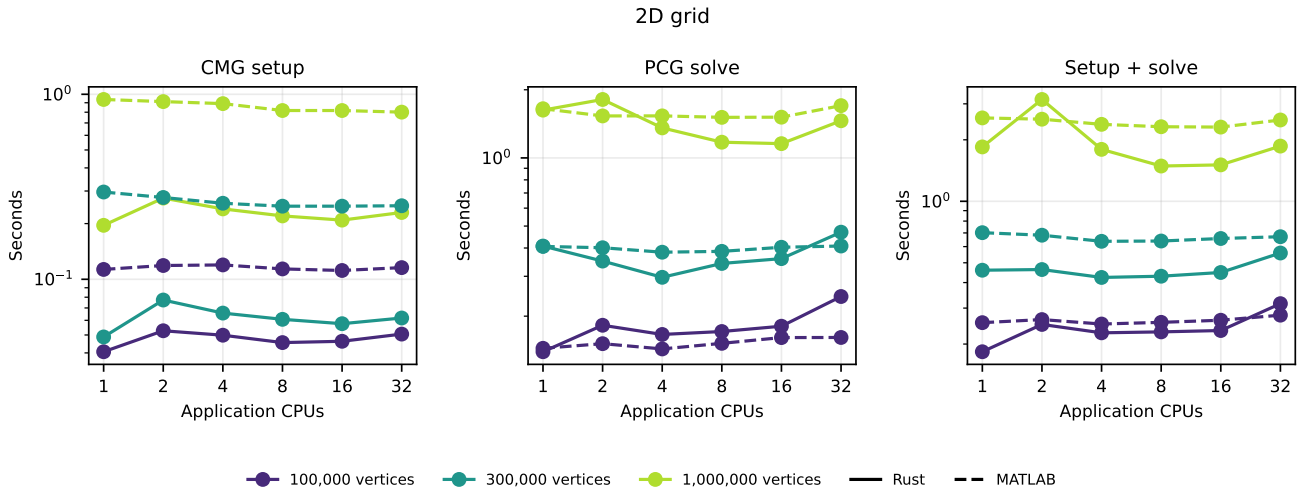

Figure 9: CPU scaling: two-dimensional grid.

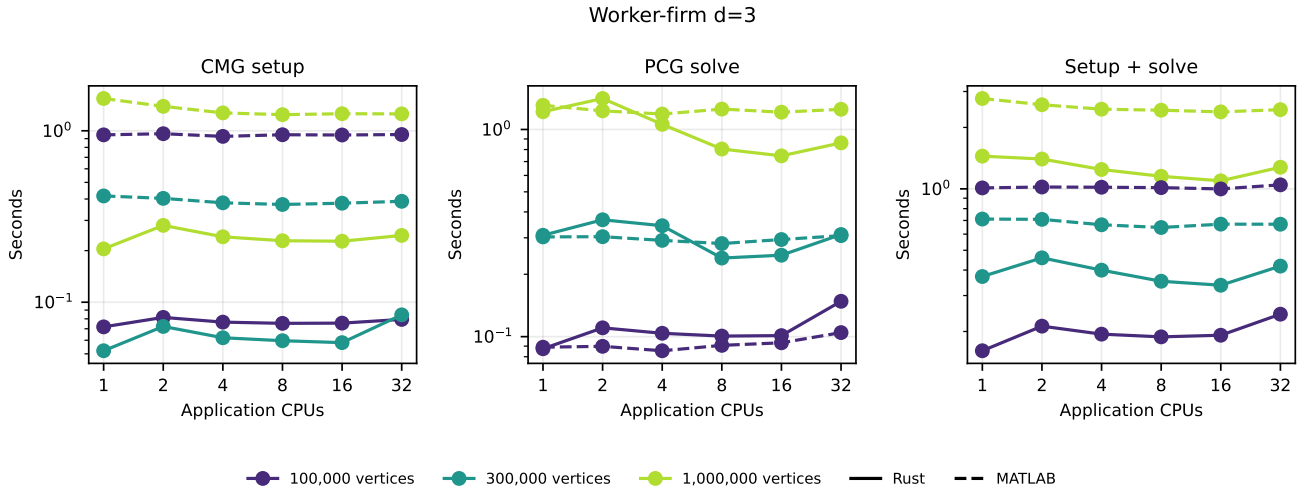

Figure 10: CPU scaling: worker-firm degree 3.

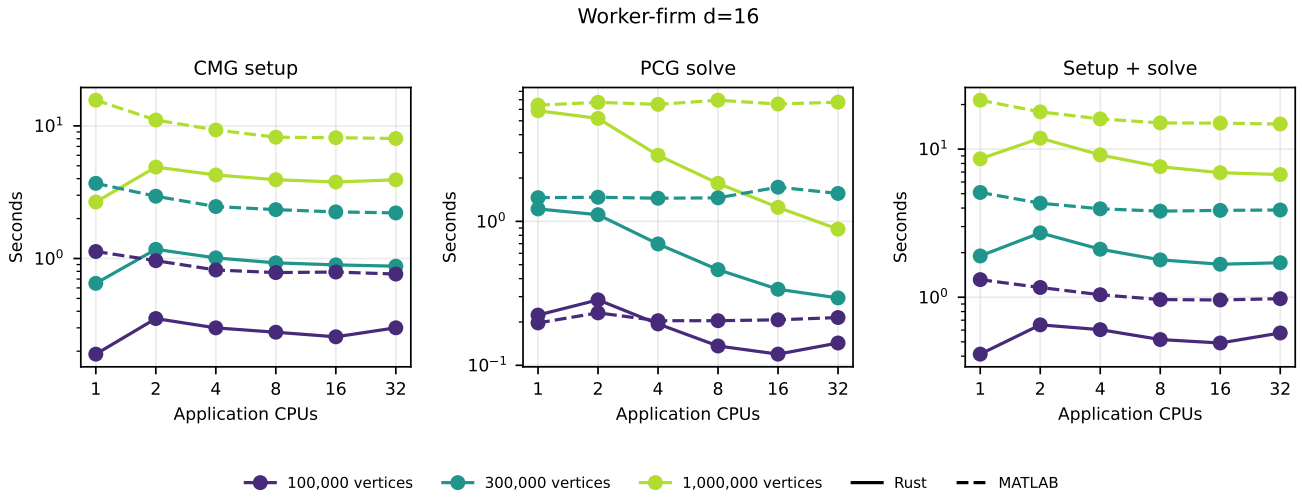

Figure 11: CPU scaling: worker-firm degree 16.

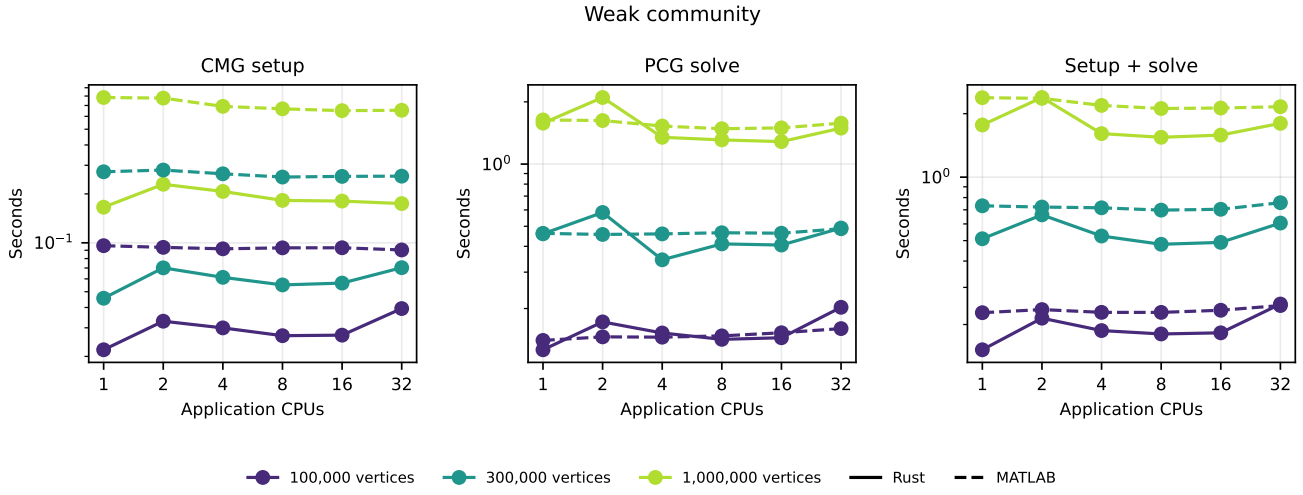

Figure 12: CPU scaling: weak-community graph.

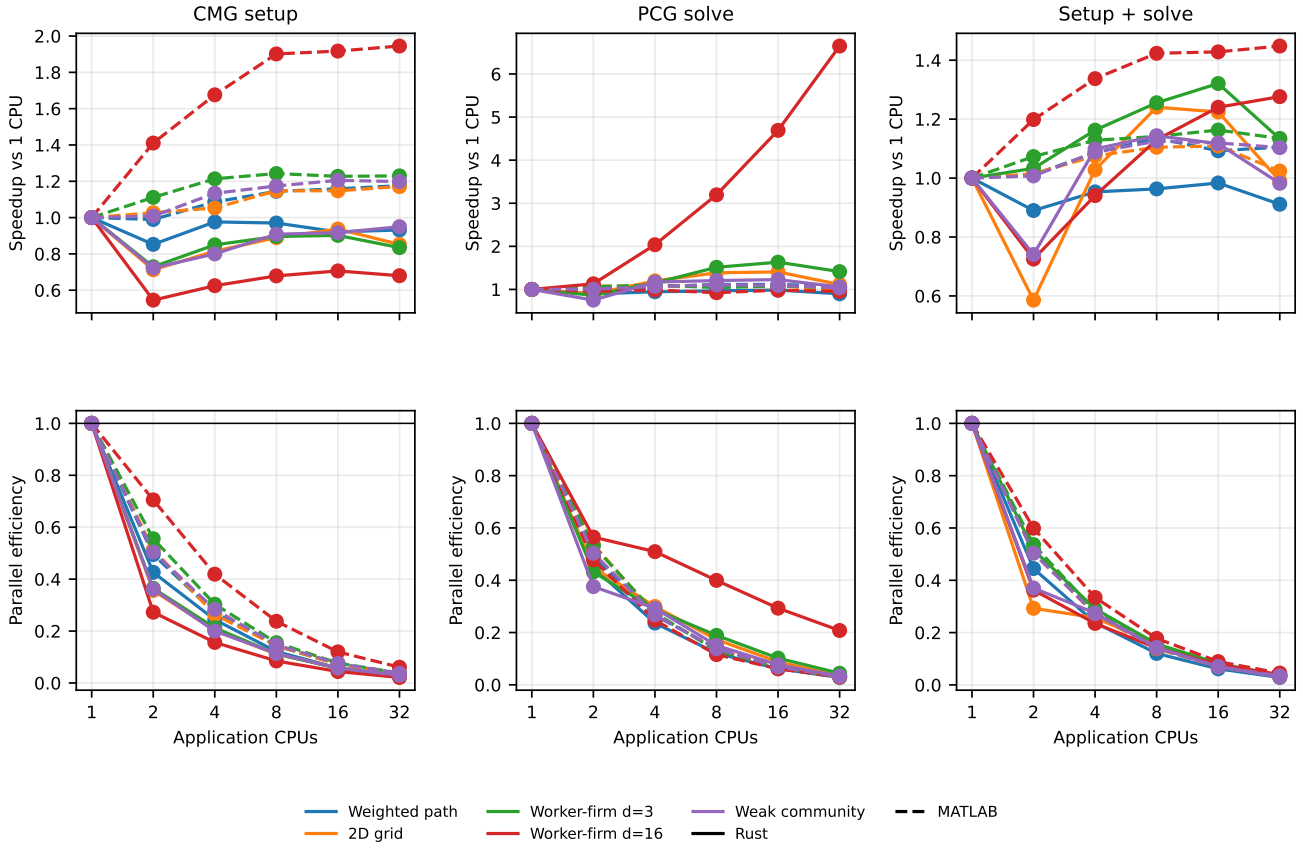

Figure 13: Speedup and parallel efficiency relative to one application CPU for one-million-vertex cases.

Batch16 SCC results pending

Worker-firm d=3 and d=16; 300,000 and 1,000,000 vertices

Figure 14: Compact sixteen-RHS supplement. Times are normalized per right-hand side after one reused setup.

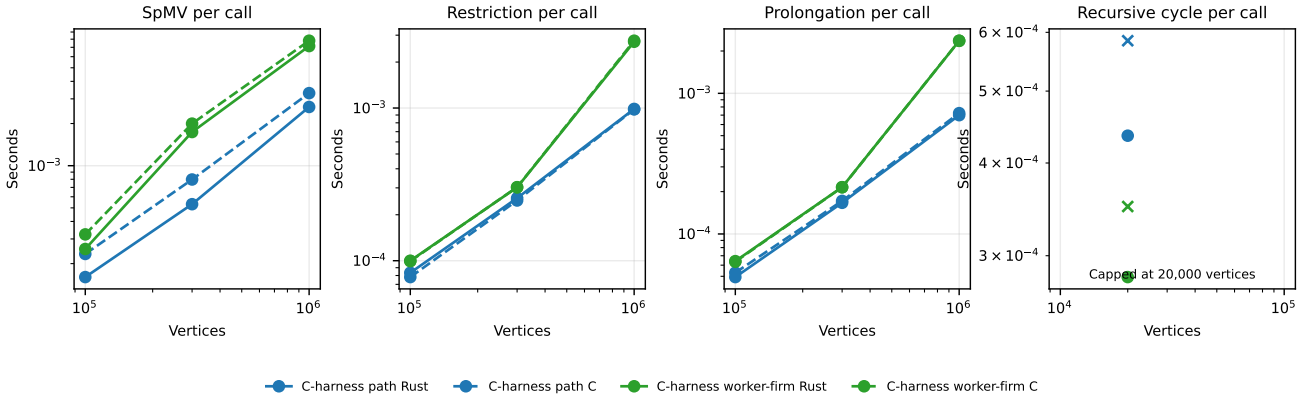

Figure 15: Scientifically bounded standalone C-kernel comparison. Large cases cover SpMV and projection only; recursive-cycle inputs are capped at 20,000 vertices.

- MATLAB PCG flag zero, a reported native relative residual, Rust backward-error certification, and independent residual diagnostics; and
- a thread-invariant hierarchy within each implementation, with cross-implementation hierarchy differences retained rather than assumed away.

Build-path, transport, or wrapper failures were preserved and retried under new run identifiers. Algorithmic warnings, convergence differences, timeouts, or memory failures would remain scientific results rather than being silently rerun.

The main array satisfied these criteria after the narrowly scoped source-identity repair described above. The correction is treated as derived bookkeeping, not as a rerun or replacement of experimental measurements. Its machine-readable receipt and the immutable raw run make that judgment independently auditable.

10 Limitations and expansion protocol

This first phase uses deterministic synthetic graphs, three measured repetitions, one homogeneous CPU model, default release/MEX builds, and maximum size one million vertices. The compact repetition count supports directional engineering conclusions but not formal confidence intervals. MATLAB and Rust retain different native stopping conventions and hierarchy implementations; iteration and residual diagnostics must accompany raw time comparisons. Peak RSS is process-level rather than a decomposition of every retained allocation. MATLAB's CPU limit does not imply that every sparse primitive uses every permitted thread. The CPU-scaling curves show that a 32-slot allocation does not imply 32-fold application speedup, particularly when hierarchy setup dominates.

An expansion should create a new UTC run directory and preserve this run unchanged. Recommended extensions are: larger vertex counts until time or memory limits are observed; additional community strength and degree sweeps; five or more repetitions for uncertainty summaries; a second CPU generation; and a separately labeled native-CPU build sensitivity. The same binary fixture format, input hashes, source/toolchain manifests, scheduler receipts, validation program, and plotting script should be reused. New failures must be plotted explicitly, and any protocol change should be versioned in the machine-readable schema.

11 Reproduction

The maintained entry points are `benchmarks/scc/bootstrap.sh`, `submit.sh`, `run_array.sh`, `validate_run.py`, `benchmarks/analysis/repair_source_identity.py`, and `benchmarks/analysis/summarize.py`. Detailed commands and archive conventions are in `benchmarks/README.md`. The compact latest record lives at `.ci/performance/scc-latest.json`; raw repetitions and scheduler evidence intentionally remain outside the maintained source tree.